



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2213 — Lógica para Ciencia de la Computación — 2024-01

Interrogación 2

Duración: 2:30 hrs.

Alumno/a:

Observación: Cada pregunta tiene una nota de 1 a 7 (hay un punto base). La nota final es el promedio de las 3 preguntas. Dentro de cada pregunta, cada ítem vale lo mismo.

Pregunta 1

1. Asuma que $P \neq NP$. Demuestre que si L y L' son NP-completos, entonces $L \cap L'$ no es necesariamente NP-completo.
2. Demuestre que si L y L' están en RE, entonces $L \cup L'$ también está en RE.

Solución

Parte 1

Consideremos los lenguajes

$$L = \{0\#w : w \in \text{SAT}\}$$

$$L' = \{1\#w : w \in \text{SAT}\}$$

Notemos que ambos son NP-completos, ya que podemos tomar las reducciones de SAT $f(w) = 0\#w$ para L y $g(w) = 1\#w$ para L' . Además su intersección es el lenguaje vacío, que se puede decidir en tiempo polinomial, por lo tanto $L \cap L' \in P$. Si P y NP son distintos, entonces un lenguaje en P no puede ser NP-completo, en particular $L \cap L'$ no es NP-completo.

Parte 2

Sean L y L' en RE, entonces sabemos que existen máquinas de Turing M y M' que aceptan estos lenguajes. Ahora podemos construir N como la máquina de dos cintas tal que

1. Se copia el input en la segunda cinta.
2. En los pasos pares se ejecuta M en la primera cinta
3. En los pasos impares se ejecuta M' en la segunda cinta
4. Si se acepta el input en cualquiera de las dos cintas, N acepta

Claramente esta máquina acepta cuando el input está en L o en L' y si el input no está en ninguno de los dos lenguajes, entonces rechaza o no se detiene nunca, por lo tanto $L(N) = L \cup L'$. Es decir, $L \cup L'$ está en RE.

Cada parte tiene 3 puntos. En la primera, se da 1.5 puntos por elegir correctamente los lenguajes y 1.5 por la justificación (no tiene por qué ser muy formal, basta que mencionen que los lenguajes son NP-completos y mencionen que la intersección no lo es). En la segunda, se da 1.5 puntos por construir correctamente la máquina y 1.5 puntos por explicar por qué acepta $L \cup L'$ (si se construye la máquina de manera que primero ejecuta M y después M' , entonces dar 1 punto).

Pregunta 2

Considere el siguiente lenguaje sobre $\{0, 1\}$:

$$\text{PAR} = \{\text{cod}(M) \mid M \text{ es MT, } L(M) \neq \emptyset, \text{ y todas las palabras en } L(M) \text{ tienen largo par}\}.$$

Demuestre que PAR es indecidible, mediante una reducción desde Halting. (Es decir, $\text{Halting} \leq_m \text{PAR}$.)

Solución

Para probar que PAR es indecidible basta con reducir de otro problema indecidible, en este caso, será desde Halting. Consideremos $f : A^* \rightarrow A^*$ tal que,

1. Si w no está en el formato $(\text{cod}(M), z)$, entonces $f(w) = 0 \notin \text{PAR}$.
2. Si $w = (\text{cod}(M), z)$, entonces contruimos la máquina $N_{M,z}$ tal que dado un input u la máquina funciona de la siguiente manera
 - a) Si el largo de u es impar, entonces la máquina rechaza.
 - b) Si el largo de u es par, entonces borramos el input y ejecutamos M sobre z . Si se detiene, aceptar.

Entonces definimos $f(w) = \text{cod}(N_{M,z})$

Esta máquina es computable, ya que solo requiere verificar el largo del input y ejecutar una máquina de la que tenemos el código. Probemos ahora que $w \in \text{Halting}$ si y solo si $f(w) \in \text{PAR}$.

($\Rightarrow$) Si $w \in \text{Halting}$, entonces $w = (\text{cod}(M), z)$ y la ejecución de M sobre z se detiene. Por lo tanto, $N_{M,z}$ siempre se detendrá cuando se le da un input par y aceptará, pero rechazará los inputs impares. Es decir, $L(N_{M,z}) = \{w : w \text{ es de largo par}\}$ y $\text{cod}(N_{M,z}) \in \text{PAR}$.

($\Leftarrow$) Si $w \notin \text{Halting}$, entonces hay dos casos:

1. w no está en el formato adecuado. Entonces $f(w) = 0 \notin \text{PAR}$.
2. $w = (\text{cod}(M), z)$ pero la ejecución de M no se detiene sobre z . Entonces tenemos que N rechaza todos los inputs impares y nunca se detiene con los inputs pares, es decir $L(N_{M,z}) = \emptyset$ y $f(w) = \text{cod}(N_{M,z}) \notin \text{PAR}$.

En conclusión, la función f es una reducción de Halting a PAR, de lo que podemos concluir que PAR no es decidable.

La mitad del puntaje se da por construir correctamente la reducción y la otra mitad por la demostración de la doble implicancia.

En caso de construir la máquina vista en clases, es decir $N_{M,z}$ hace lo siguiente: sobre entrada u , ignorar u y simular M sobre w . Si se detiene, aceptar. Entonces dar 2 puntos.

En caso de probar lo pedido, pero no mediante una reducción, dar máximo 3 puntos.

Pregunta 3

Dado un conjunto de conjuntos $\mathcal{S}$, decimos que $\mathcal{C} \subseteq \mathcal{S}$ es un *set cover* de $\mathcal{S}$ si se cumple:

$$\bigcup_{C \in \mathcal{C}} C = \bigcup_{S \in \mathcal{S}} S.$$

Por ejemplo, considere $\mathcal{S} = \{\{1, 2, 3\}, \{2, 4, 5\}, \{1, 4\}, \{3, 5\}, \{1, 3, 4\}\}$. Tenemos que $\bigcup_{S \in \mathcal{S}} S = \{1, 2, 3, 4, 5\}$. En este caso, se cumple que $\mathcal{C} \subseteq \mathcal{S}$, dado por $\mathcal{C} = \{\{1, 2, 3\}, \{1, 4\}, \{3, 5\}\}$ es un set cover de $\mathcal{S}$, ya que $\bigcup_{C \in \mathcal{C}} C = \{1, 2, 3, 4, 5\}$. Otro posible set cover es $\mathcal{C} = \{\{2, 4, 5\}, \{1, 3, 4\}\}$. Por otra parte, si tomamos $\mathcal{C} = \{\{1, 2, 3\}, \{1, 4\}, \{1, 3, 4\}\}$, no obtenemos un set cover de $\mathcal{S}$, ya que $\bigcup_{C \in \mathcal{C}} C = \{1, 2, 3, 4\} \neq \{1, 2, 3, 4, 5\}$.

Ojo: En general, los conjuntos de $\mathcal{S}$ pueden contener cualquier tipo de objeto discreto, por ejemplo, números (como en el ejemplo de arriba), nodos de un grafo, aristas de un grafo, etc...

Considere el siguiente problema:

SET-COVER = $\{(\mathcal{S}, k) \mid \mathcal{S} \text{ es un conjunto de conjuntos, } k \geq 1, \text{ y existe un set cover } \mathcal{C} \text{ de } \mathcal{S} \text{ tal que } |\mathcal{C}| = k\}$.

Demuestre que SET-COVER es NP-completo.

(*Hint:* para el NP-hardness utilice VERTEX-COVER. Recuerde que en el problema VERTEX-COVER, dado (G, k) , hay que decidir acaso G tiene un vertex cover D con $|D| = k$. Recuerde también que un vertex cover de G es un conjunto de nodos D tal que para toda arista $\{u, v\}$, se cumple $u \in D$ o $v \in D$.)

Solución

Para probar que SET-COVER es NP-completo, hay que probar que está en NP y reducir polinomialmente de un lenguaje NP-completo a SET-COVER.

SET-COVER $\in$ NP

Podemos construir una máquina de Turing M tal que:

1. Dados $((\mathcal{S}, k), \mathcal{C})$ se verifica que $\mathcal{C}$ tenga k conjuntos y $\mathcal{C} \subseteq \mathcal{S}$. Si no es así, se rechaza.
2. Para cada elemento de los conjuntos de $\mathcal{S}$ se revisa si está contenido en algún conjunto de $\mathcal{C}$. Si existe alguno que no, entonces se rechaza. Si todos los elementos de los conjuntos de $\mathcal{S}$ están en algún conjunto de $\mathcal{C}$, se acepta.

Esta máquina solo acepta inputs $((\mathcal{S}, k), \mathcal{C})$ tales que $\mathcal{C}$ es un set cover de $\mathcal{S}$ con k conjuntos y funciona en tiempo polinomial, ya que verificar el largo de $\mathcal{C}$ es lineal y luego verificar la pertenencia de los elementos de $\mathcal{S}$ en $\mathcal{C}$ es cuadrático.

Además, notemos que si $((\mathcal{S}, k), \mathcal{C})$ es aceptado, entonces el largo de $\mathcal{C}$ es menor que el largo de $\mathcal{S}$, ya que es un subconjunto, por lo tanto tenemos un verificador polinomial para SET-COVER. Es decir, SET-COVER está en NP.

SET-COVER es NP-hard

Construiremos la reducción de VERTEX-COVER. Sea $w \in A^*$, si w no es un grafo, definimos $f(w) = 0 \notin$ SET-COVER. Si $w = (G, k)$ un grafo con vértices V y aristas E , entonces definimos $w = (\mathcal{S}_G, k)$ tal que $\mathcal{S} = \{S_v : v \in V\}$ con

$$S_v = \{e \in E : e \text{ es una arista con un extremo en el vértice } v\}$$

Esto se puede construir en tiempo polinomial ya que solo requiere recorrer las aristas una vez para cada vértice e ir copiando las aristas correspondientes. Entonces solo queda probar que $(G, k) \in$ VERTEX-COVER si y solo si $(\mathcal{S}_G, k) \in$ SET-COVER.

($\Rightarrow$) Sea $(G, k) \in \text{VERTEX-COVER}$, entonces existe una colección de vértices $v_1, \dots, v_k$ tales que todas las aristas inciden en alguno de ellos. Entonces tenemos que todas las aristas deben estar en alguno de los conjuntos $S_{u_1}, \dots, S_{u_k}$, es decir, esa colección es un set cover de $\mathcal{S}_G$ con k conjuntos, ie. $(\mathcal{S}_G, k) \in \text{SET-COVER}$.

($\Leftarrow$) Si $(G, k) \notin \text{VERTEX-COVER}$, consideremos k conjuntos en $\mathcal{S}_G$, entonces tenemos un vértice asociado a cada uno de los conjuntos $v_1, \dots, v_k$. Como no hay vertex cover de G con k vértices, existe una arista e tal que no está conectada a ninguno de los vértices, es decir, e no estará en ninguno de los conjuntos S_{u_i} para $i = 1, \dots, k$. Es decir, no puede haber un set cover de $\mathcal{S}_G$ con k conjuntos y $(\mathcal{S}_G, k) \notin \text{SET-COVER}$.

Por lo tanto, SET-COVER es NP-completo.

Dar 2 puntos por probar que el lenguaje está en NP (puede ser también construyendo la máquina no determinista y 4 puntos por probar que es NP-hard, dos de ellos por la función y uno por cada lado de la implicancia.